CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 10: Hash Tables: Linear Probing, Chaining, and Double Hashing

Practical Aim: To implement hash tables using different collision handling strategies and develop the ability to reason about why collisions occur, how different strategies resolve them, and what trade-offs each approach involves.

Problem 1:

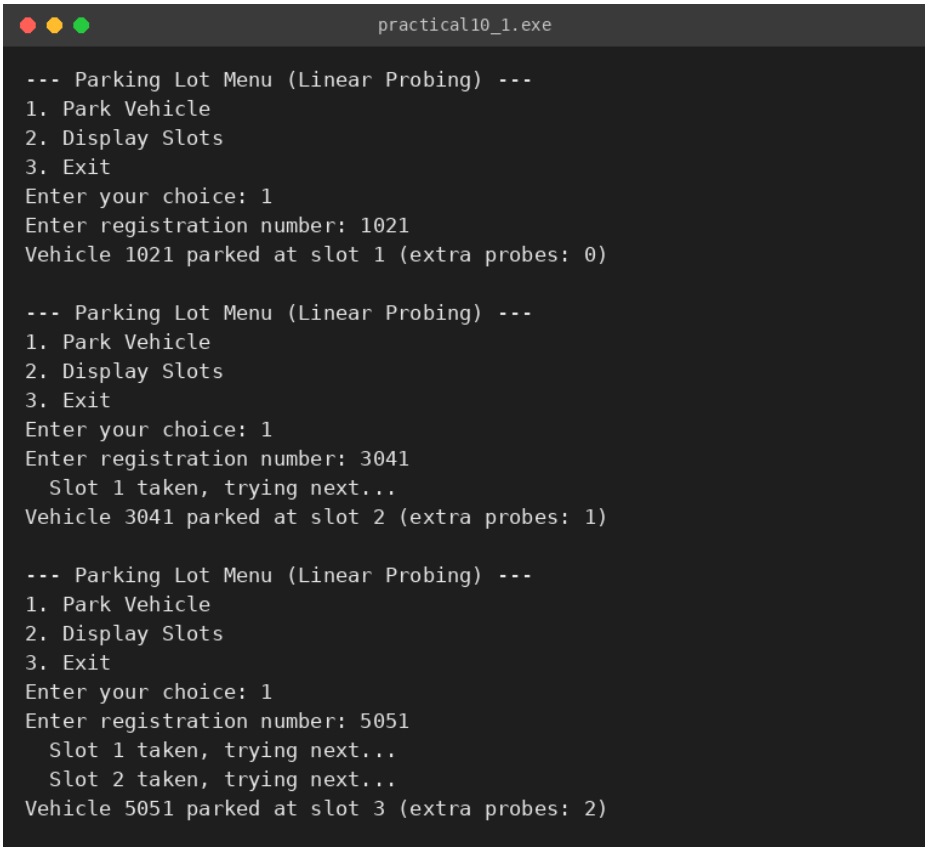
A small parking lot has exactly 10 numbered slots (0 to 9). When a vehicle arrives, its registration number is used to assign it a slot by taking the last digit of the number. If that slot is already taken, the vehicle moves to the next available slot in order, wrapping around if necessary. Given a sequence of vehicle registrations, assign each to a slot using this rule and display the final state of all ten slots.

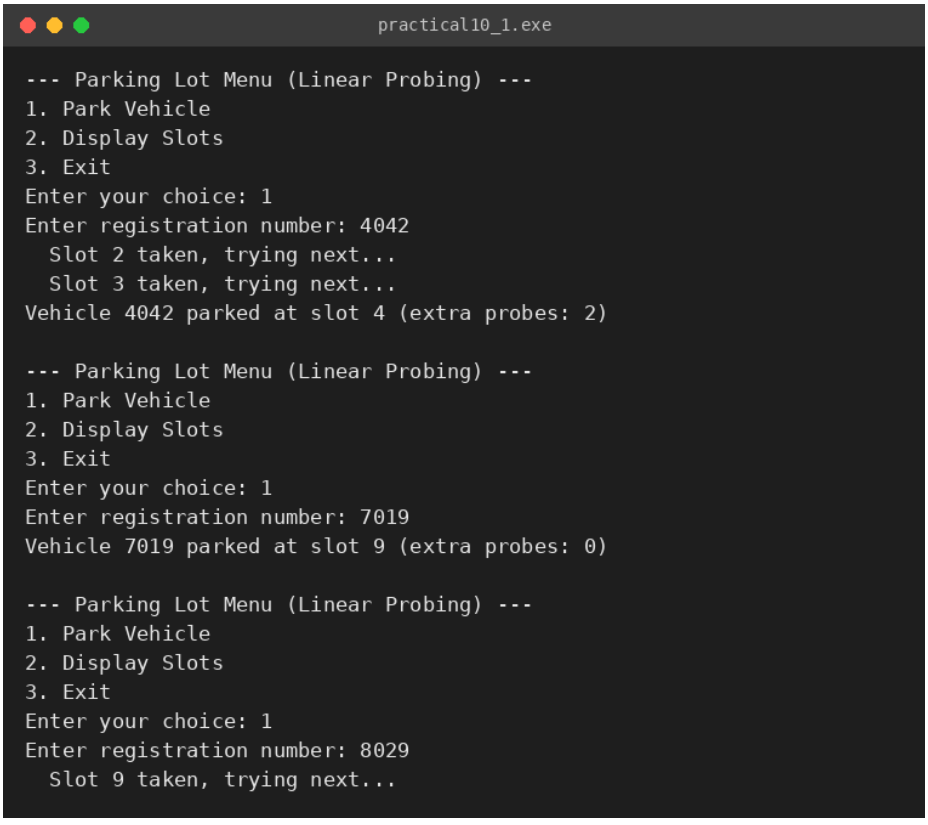
Approach: The hash function is reg % 10 (the last digit). If the slot is occupied (collision), the program moves to the next slot using (slot + 1) % 10, so it wraps around from slot 9 to slot 0, until a free slot is found. When the lot is nearly full and many vehicles land on the same slot, the probe walks through a long run of taken slots (the sample run shows 2100 needing 5 extra probes). The search could loop indefinitely only if the table were completely full, because there would then be no free slot to stop at; the program therefore counts the occupied slots and refuses to park a vehicle when all 10 slots are taken.

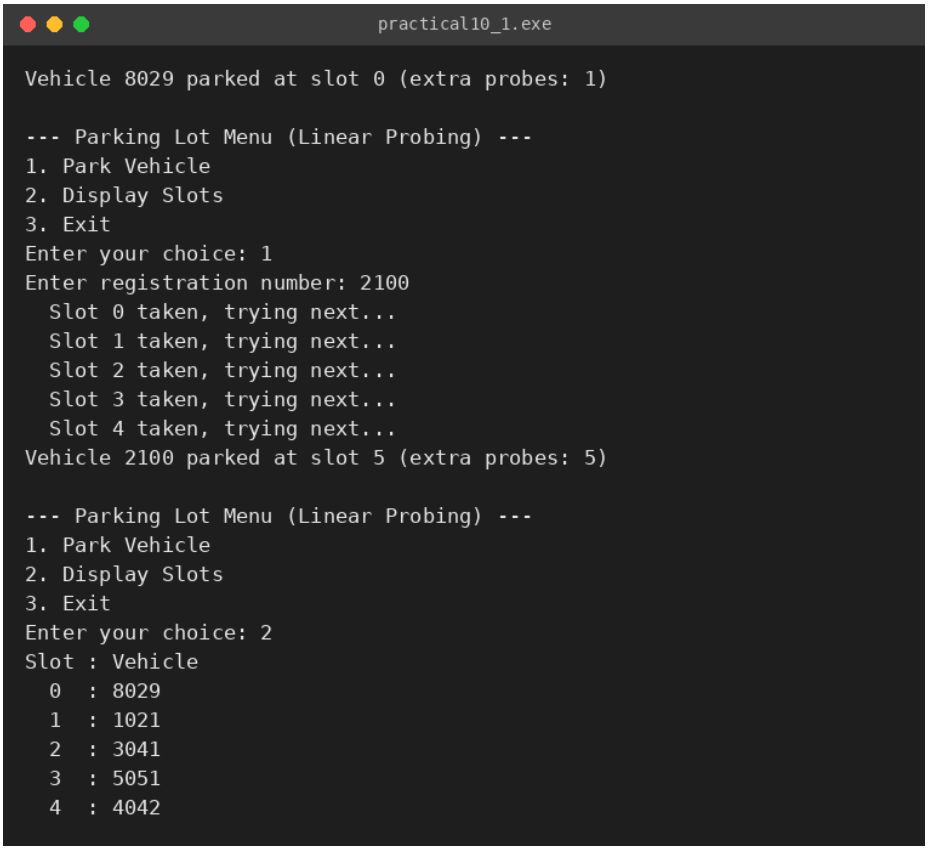
For Code:

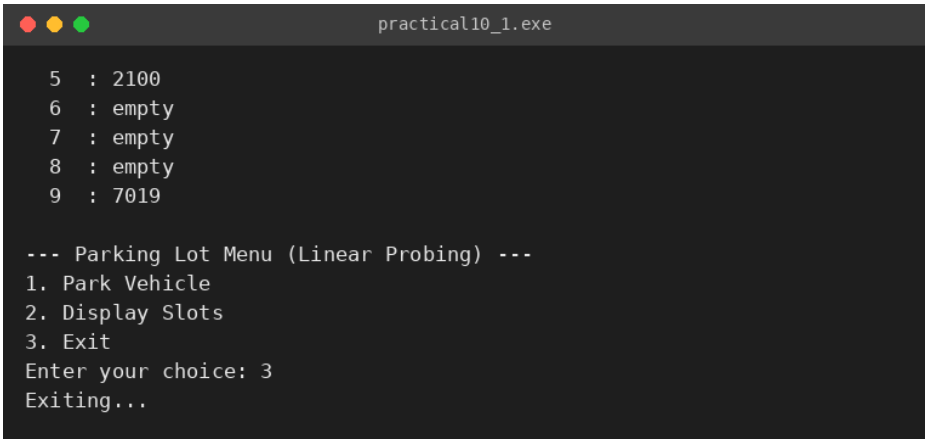
https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-10/practical10_1.cpp

Output :


practical10_1.exe
--- Parking Lot Menu (Linear Probing) ---
1. Park Vehicle
2. Display Slots
3. Exit
Enter your choice: 1
Enter registration number: 1021
Vehicle 1021 parked at slot 1 (extra probes: 0)
--- Parking Lot Menu (Linear Probing) ---
1. Park Vehicle
2. Display Slots
3. Exit
Enter your choice: 1
Enter registration number: 3041
Slot 1 taken, trying next...
Vehicle 3041 parked at slot 2 (extra probes: 1)
--- Parking Lot Menu (Linear Probing) ---
1. Park Vehicle
2. Display Slots
3. Exit
Enter your choice: 1
Enter registration number: 5051
Slot 1 taken, trying next...
Slot 2 taken, trying next...
Vehicle 5051 parked at slot 3 (extra probes: 2)


practical10_1.exe
--- Parking Lot Menu (Linear Probing) ---
1. Park Vehicle
2. Display Slots
3. Exit
Enter your choice: 1
Enter registration number: 4042
Slot 2 taken, trying next...
Slot 3 taken, trying next...
Vehicle 4042 parked at slot 4 (extra probes: 2)
--- Parking Lot Menu (Linear Probing) ---
1. Park Vehicle
2. Display Slots
3. Exit
Enter your choice: 1
Enter registration number: 7019
Vehicle 7019 parked at slot 9 (extra probes: 0)
--- Parking Lot Menu (Linear Probing) ---
1. Park Vehicle
2. Display Slots
3. Exit
Enter your choice: 1
Enter registration number: 8029
Slot 9 taken, trying next...





Problem 2:

A library uses a system with 10 shelves numbered 0 to 9. Each book's code is used to decide which shelf it belongs to by taking the code modulo 10. Unlike the parking lot, each shelf can hold multiple books stacked together — when two books map to the same shelf, they are simply added to that shelf's stack. Given a sequence of book codes, place each on the correct shelf and display the final contents of all shelves.

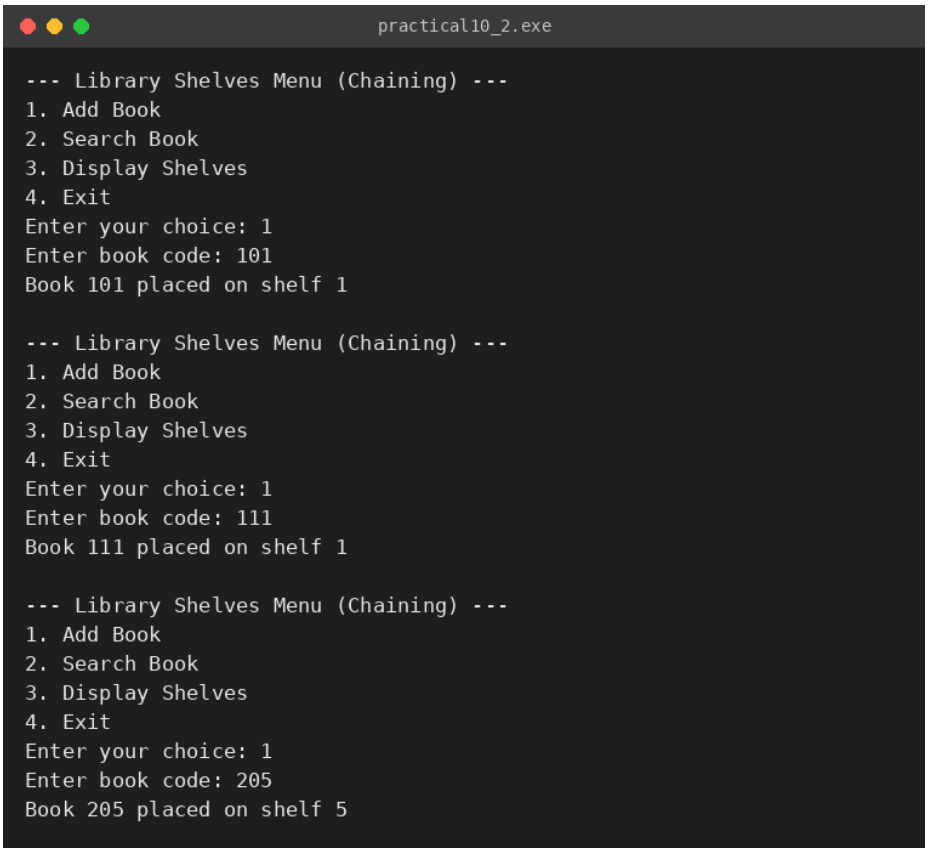
Approach: This is separate chaining: every shelf is the head of a linked list. A book is placed on shelf code % 10 by inserting a new node at the front of that shelf's list, so a collision needs no probing and never moves other books. Searching a book means jumping to its shelf and walking

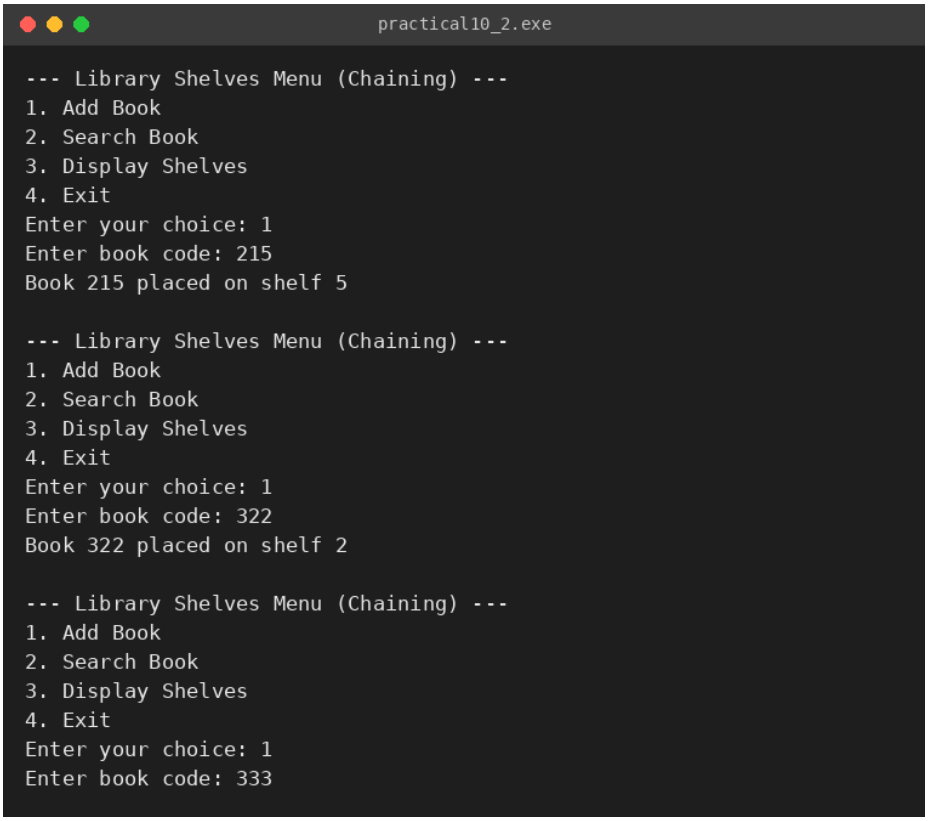
down that list, so the search time depends on the length of the chain: as more and more books pile onto one shelf the search time grows from O(1) towards O(n). All books end up on the same shelf when the hash function maps them all to the same index, e.g. when all codes are multiples of 10.

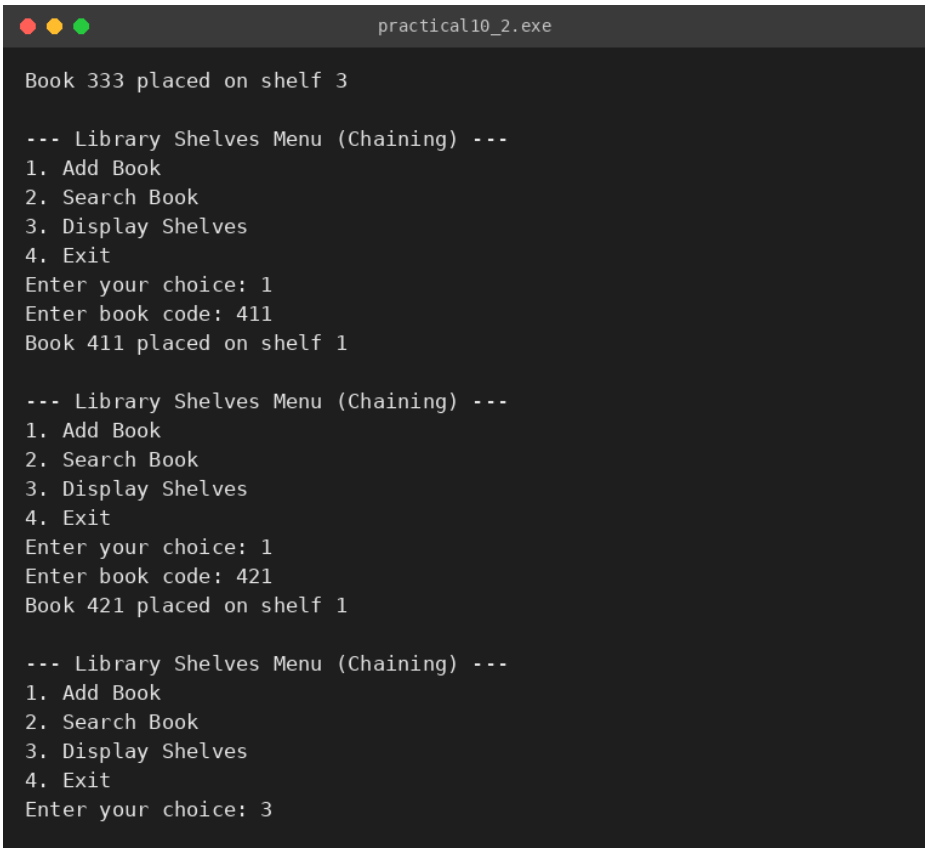
For Code:

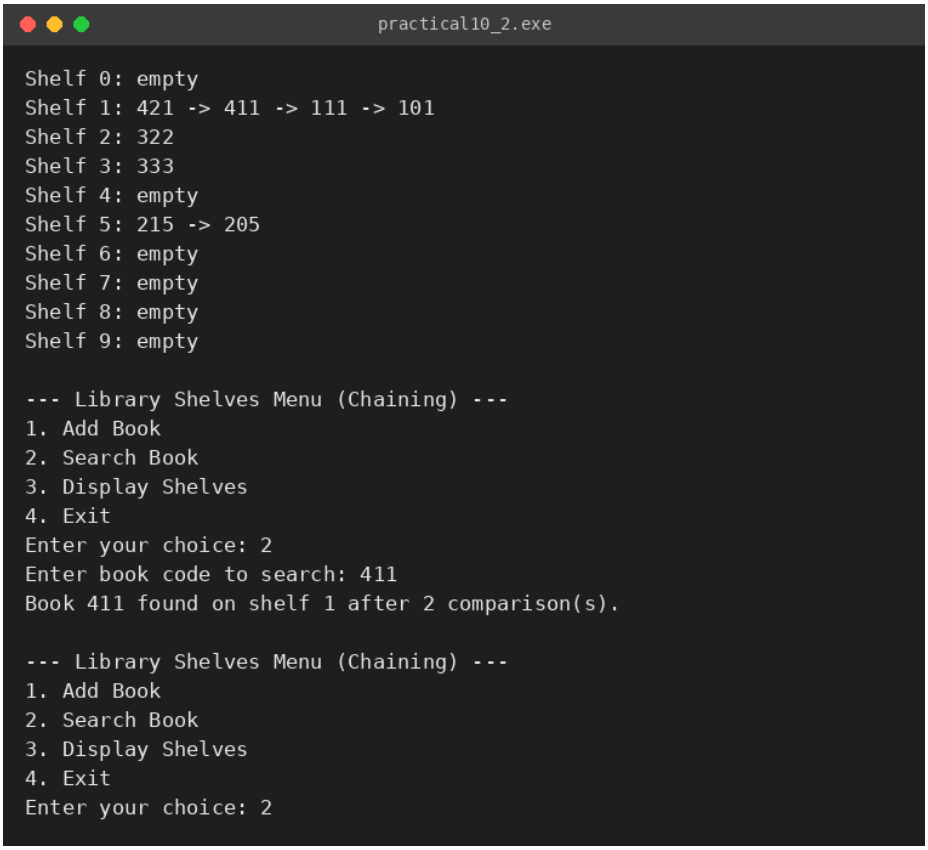
https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-10/practical10_2.cpp

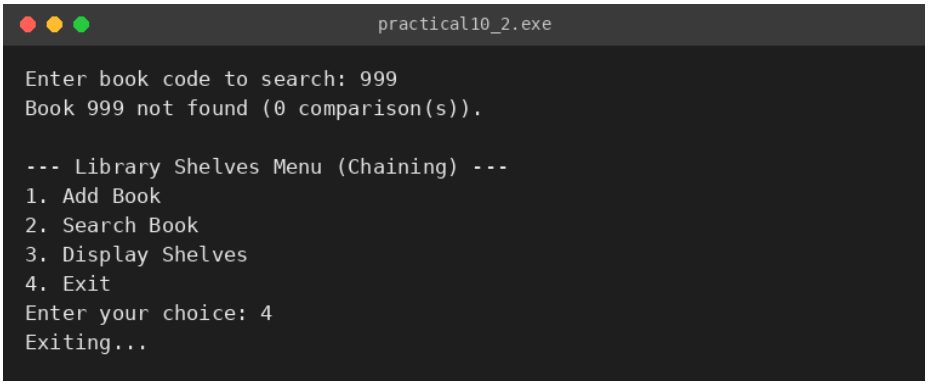
Output :




practical10_2.exe
--- Library Shelves Menu (Chaining) ---
1. Add Book
2. Search Book
3. Display Shelves
4. Exit
Enter your choice: 1
Enter book code: 215
Book 215 placed on shelf 5
--- Library Shelves Menu (Chaining) ---
1. Add Book
2. Search Book
3. Display Shelves
4. Exit
Enter your choice: 1
Enter book code: 322
Book 322 placed on shelf 2
--- Library Shelves Menu (Chaining) ---
1. Add Book
2. Search Book
3. Display Shelves
4. Exit
Enter your choice: 1
Enter book code: 333


practical10_2.exe
Book 333 placed on shelf 3
--- Library Shelves Menu (Chaining) ---
1. Add Book
2. Search Book
3. Display Shelves
4. Exit
Enter your choice: 1
Enter book code: 411
Book 411 placed on shelf 1
--- Library Shelves Menu (Chaining) ---
1. Add Book
2. Search Book
3. Display Shelves
4. Exit
Enter your choice: 1
Enter book code: 421
Book 421 placed on shelf 1
--- Library Shelves Menu (Chaining) ---
1. Add Book
2. Search Book
3. Display Shelves
4. Exit
Enter your choice: 3





Problem 3:

A university assigns student IDs to storage slots using a two-step rule. The first rule determines the initial slot. If that slot is taken, instead of simply moving one slot forward, a second rule based on the ID itself determines how many slots to jump. This means different IDs that collide will probe different sequences of slots, spreading them more evenly. Given a sequence of student IDs, assign each to a slot using both rules and display the final state of the table.

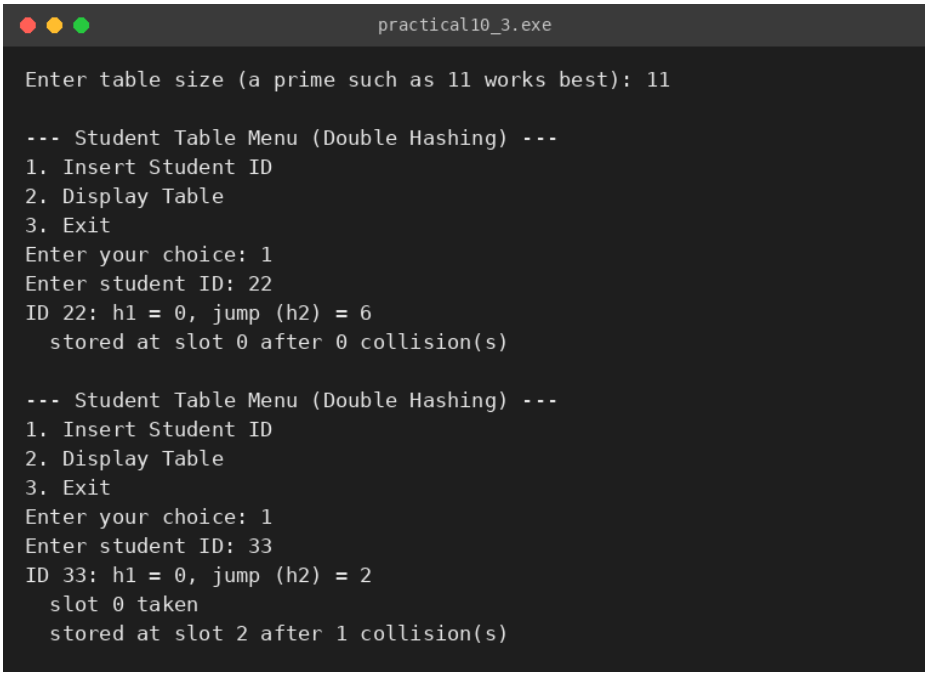
Approach: This is double hashing. The first rule is h1(id) = id % size, giving the initial slot. If it is taken, the second rule h2(id) = 7 - (id % 7) gives the jump size (always between 1 and 7, never 0), and the probe sequence is (h1 + i * h2) % size for i = 0, 1, 2, ... Different IDs therefore get different jump sizes (in the sample run IDs 33, 44, 55 and 66 all start at slot 0 but jump by 2, 5, 1

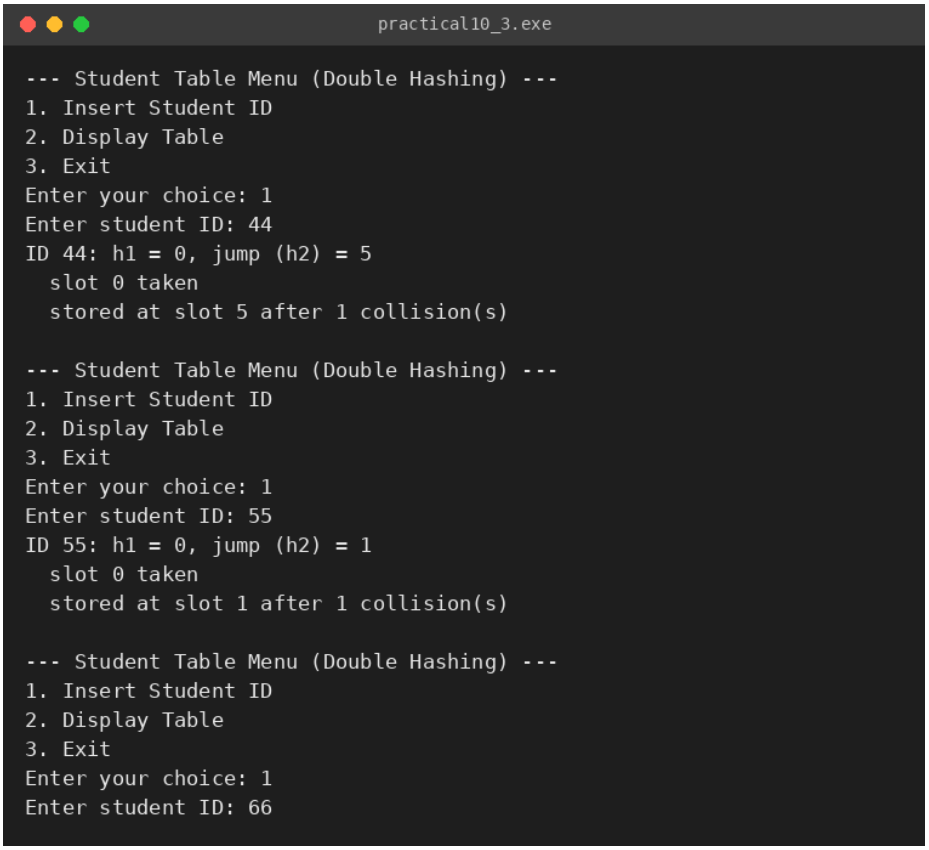
and 4). If the jump size shares a common factor with the table size (e.g. jump 2 in a table of size 10), the probe sequence only visits some of the slots and may cycle without finding a free one, so a prime table size (11 in the sample run) is used, and the probing is limited to 'size' attempts. Unlike linear probing, where every colliding key follows the same one-step path, double hashing spreads collisions over many different paths.

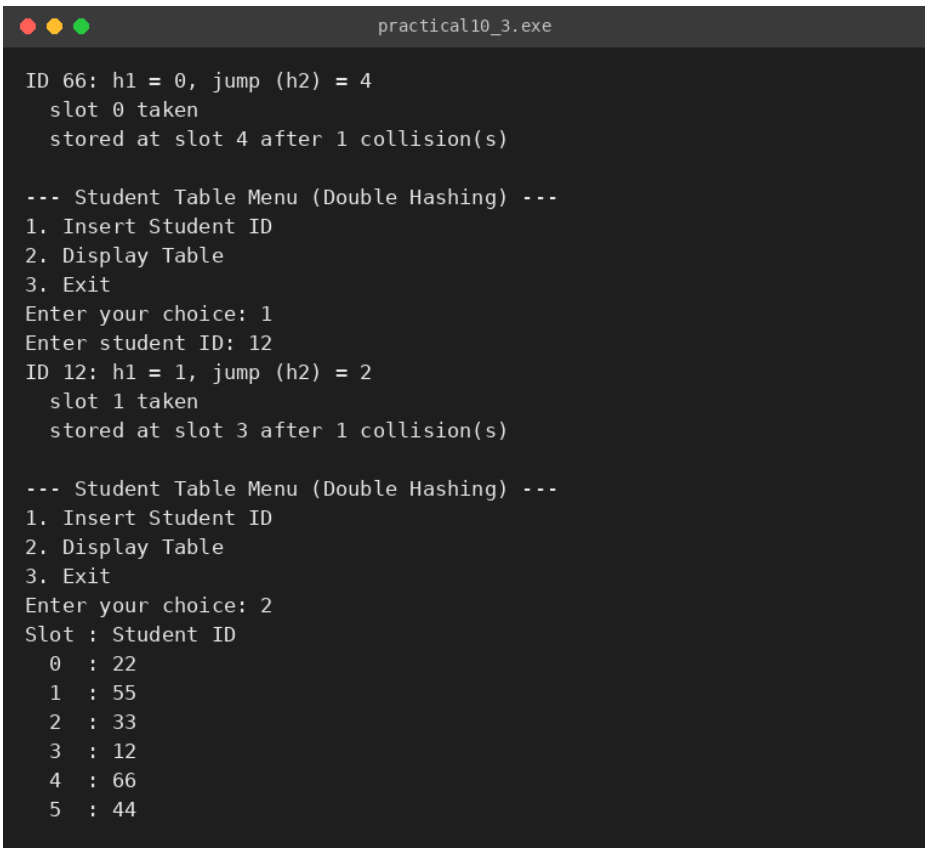
For Code:

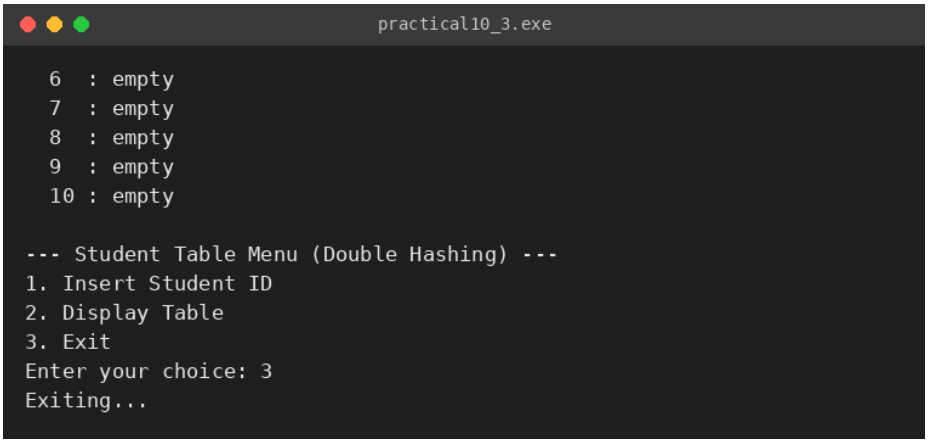
https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-10/practical10_3.cpp

Output :




practical10_3.exe
--- Student Table Menu (Double Hashing) ---
1. Insert Student ID
2. Display Table
3. Exit
Enter your choice: 1
Enter student ID: 44
ID 44: h1 = 0, jump (h2) = 5
slot 0 taken
stored at slot 5 after 1 collision(s)
--- Student Table Menu (Double Hashing) ---
1. Insert Student ID
2. Display Table
3. Exit
Enter your choice: 1
Enter student ID: 55
ID 55: h1 = 0, jump (h2) = 1
slot 0 taken
stored at slot 1 after 1 collision(s)
--- Student Table Menu (Double Hashing) ---
1. Insert Student ID
2. Display Table
3. Exit
Enter your choice: 1
Enter student ID: 66


practical10_3.exe
ID 66: h1 = 0, jump (h2) = 4
slot 0 taken
stored at slot 4 after 1 collision(s)
--- Student Table Menu (Double Hashing) ---
1. Insert Student ID
2. Display Table
3. Exit
Enter your choice: 1
Enter student ID: 12
ID 12: h1 = 1, jump (h2) = 2
slot 1 taken
stored at slot 3 after 1 collision(s)
--- Student Table Menu (Double Hashing) ---
1. Insert Student ID
2. Display Table
3. Exit
Enter your choice: 2
Slot : Student ID
0 : 22
1 : 55
2 : 33
3 : 12
4 : 66
5 : 44



KEY QUESTIONS:

Problem 1: If five consecutive vehicle registrations all end in the same digit, trace where each one gets placed in the parking lot. Now explain why this clustering effect is a structural weakness of this collision strategy — not just for this input, but in general.

Problem 2: Suppose all book codes in Problem 2 are multiples of 10. Without tracing through the code, predict what the final shelf layout would look like and explain why. What does this tell you about the relationship between the hash function and the input data?

Problem 3: In Problem 3, what would happen if the second rule always produced a jump size of zero? Trace through the consequence and explain how this failure is related to the choice of the second hash function rather than the data itself.

ANSWERS:

ANS-1: Suppose five registrations end in 3 (for example 103, 213, 323, 433, 543). The first goes to slot 3, the second finds slot 3 taken and goes to slot 4, the third goes to slot 5, the fourth to slot 6 and the fifth to slot 7. So the five vehicles occupy the consecutive slots 3 to 7, with probes of 0, 1, 2, 3 and 4 extra steps.

This is primary clustering and is a structural weakness of linear probing: every collision is resolved by stepping one slot forward, so occupied slots join into long continuous blocks. Any new key that hashes anywhere inside the block must walk to its end and then extends it, so the block keeps growing and the probability of hitting it increases. As a result, probe sequences become longer and the cost of insertion and search moves from O(1) towards O(n), regardless of the particular input.

ANS-2: If all codes are multiples of 10, then code % 10 = 0 for every book, so all the books would be placed on shelf 0 as one long chain, and shelves 1 to 9 would remain empty. The table would behave like a single linked list, so a search would take O(n) instead of O(1).

The data is not wrong; the hash function does not match the data. The modulus 10 shares a factor with the pattern in the input, so only one of the ten possible results is ever used. A good hash function must spread the real input uniformly across all indices; for example using a prime table size (such as 11), or a function that uses all digits of the code, would spread these books on different shelves.

ANS-3: If the second rule always gave a jump size of zero, the probe position start + i * 0 would always be the starting slot. After a collision the program would test the same occupied slot again and again, never reaching any other slot, so the insertion would loop forever (or fail after the maximum number of attempts) even though the table still has many empty slots.

This failure comes from the choice of the second hash function, not from the data: h2 must never produce 0 for any key. That is why the program uses h2(id) = 7 - (id % 7), which always lies between 1 and 7, and also uses a prime table size so that every jump size is co-prime with the table size and the probe sequence can reach every slot.